

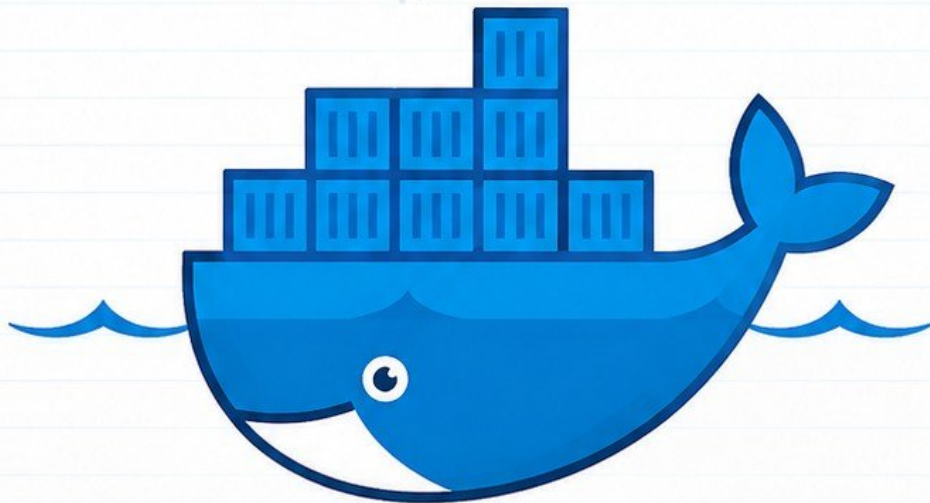


DOCKER

FOR

COMPLETE BEGINNERS

YOUR STEP-BY-STEP GUIDE TO CONTAINERS,
IMAGES, AND REAL-WORLD APPLICATIONS



LEARN
DOCKER BASICS



BUILD
IMAGES



MANAGE
CONTAINERS



DEPLOY
APPLICATIONS

FROM ZERO TO CONFIDENT

DOCKER

EXPLAINED FROM ZERO



WHAT DOCKER ACTUALLY IS

- Docker is a platform that allows you to package an application and all its dependencies into a standard unit called a **container**, so it can run consistently anywhere.



THE PROBLEM CONTAINERS SOLVE

- Different environments cause failures.
- Dependencies clash.
- Manual setup is slow and error-prone.
- Scaling applications is hard.
- Inconsistent deployments.



APPLICATION DEPENDENCIES

Every application depends on:

- Runtime (Python, Node, Java, .NET)
- Libraries
- System tools
- Configuration files
- Environment variables

Managing all of these manually is difficult and inconsistent.



"WORKS ON MY MACHINE" PROBLEM

- It works on your machine.
- Fails on a teammate's machine.
- Fails in staging.
- Fails in production.

The root cause is environment inconsistency, not your code.



CONTAINERS VS TRADITIONAL INSTALLATION

TRADITIONAL INSTALLATION	CONTAINERS
• Install dependencies manually • Different versions on different machines • Configuration drift • Hard to reproduce environments • Slow setup and deployments	• Package everything together • Same environment everywhere • Consistent and isolated • Easy to reproduce • Fast deployment and scaling

KEY BENEFITS

- ✓ Consistency
- ✓ Isolation
- ✓ Portability
- ✓ Speed
- ✓ Scalability



WHERE DOCKER FITS IN DEVOPS

- Build once, run anywhere
- Standardizes environments across the pipeline
- Speeds up CI/CD
- Makes testing predictable
- Simplifies deployments
- Helps with scaling and high availability
- Foundation for Kubernetes and cloud-native



THE BIG IDEA

Docker packages your application with everything it needs, so it can run the same way on any machine, every time.

APP → CONTAINER → HOST

APPLICATION



Your Code

CONTAINER



Application + Dependencies
Packaged Together

HOST



OS / Infrastructure
Runs the Container



YouTube



GitHub



LinkedIn



X



Instagram



Telegram



VERIQTA

CONTAINERS VS VIRTUAL MACHINES



WHAT IS A VIRTUAL MACHINE?

- A virtual machine (VM) is a software emulation of a physical computer.
- It runs a full operating system (OS) on top of a hypervisor.
- Each VM includes OS, libraries, binaries, and applications.
- Heavier, slower to start, and uses more resources.



WHAT IS A CONTAINER?

- A container packages an application and its dependencies together.
- It shares the host OS kernel through the container runtime.
- Much lighter, faster to start, and uses fewer resources.
- Runs the same way anywhere.



HYPERVISOR

Runs and manages virtual machines. Emulates hardware for each VM. (Example: VMware, Hyper-V, KVM)



CONTAINER RUNTIME

Runs and manages containers. Shares the host OS kernel. (Example: Docker, containerd, CRI-O)



SHARED KERNEL

Containers share the host OS kernel. They do not need their own OS, which makes them lightweight.



STARTUP TIME

VMs: Minutes to start

Containers: Milliseconds to start



RESOURCE USAGE

VMs: High

Containers: Low



ISOLATION

VMs: Strong Isolation

Containers: Process-level Isolation

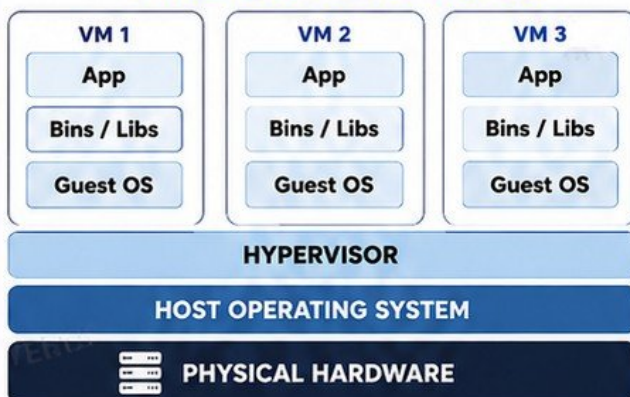


WHEN VMs STILL MAKE SENSE

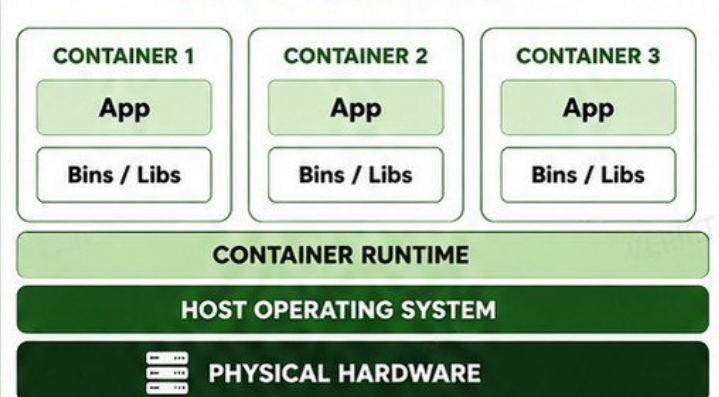
- When you need to run different operating systems (Windows + Linux together).
- When you need strong isolation for untrusted workloads.
- When you need legacy applications that require a full OS.
- When hardware-level isolation is required for compliance or security.



VIRTUAL MACHINE ARCHITECTURE



CONTAINER ARCHITECTURE



KEY TAKEAWAY

Containers are lightweight, fast, and efficient because they share the host OS kernel. VMs are heavier but provide stronger isolation and OS-level flexibility.



CONTAINERS WIN for speed, portability, and resource efficiency. VMs win for isolation and OS flexibility.



YouTube



GitHub



LinkedIn



X



Instagram



Telegram



VERIQTA

DOCKER ARCHITECTURE



DOCKER CLI

The command-line interface you use to run Docker commands.



DOCKER DAEMON

The background service that receives commands from the CLI and manages Docker objects.



DOCKER ENGINE

The core technology that builds, runs, and manages containers, images, networks, and volumes.



DOCKER HOST

The machine (physical or virtual) where Docker is installed and containers run.



IMAGES

Read-only templates used to create containers.



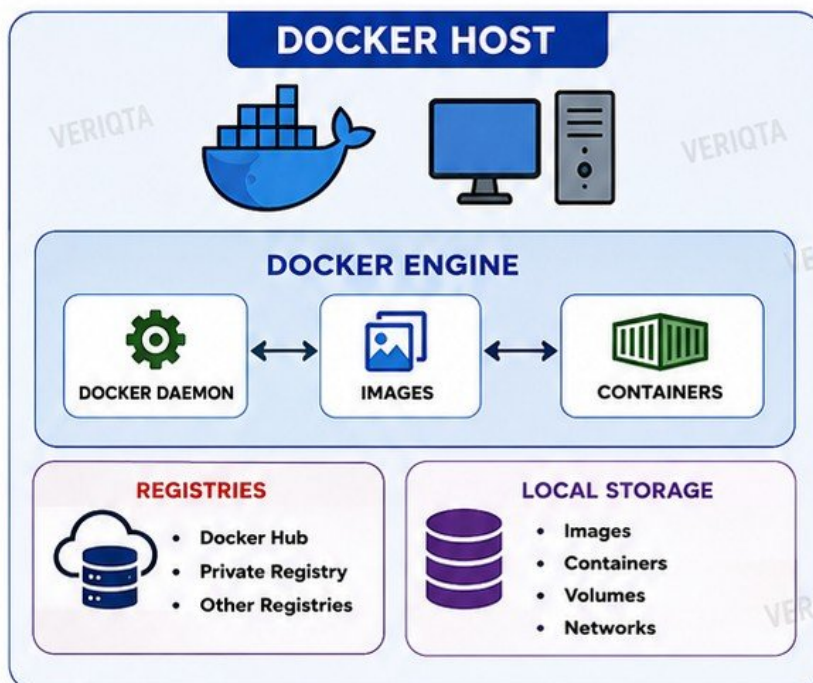
CONTAINERS

Running instances of images. They are isolated and lightweight.

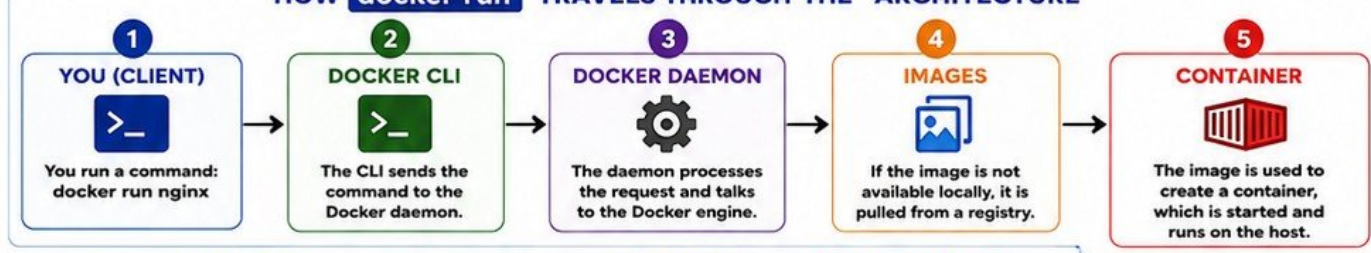


REGISTRIES

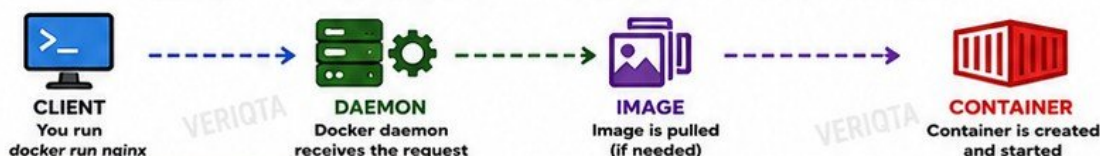
Stores images. Docker Hub is the default public registry, you can also use private registries.



HOW docker run TRAVELS THROUGH THE ARCHITECTURE



CLIENT → DAEMON → IMAGE → CONTAINER



KEY TAKEAWAY

Docker uses a client-server model. You talk to the daemon, and it builds, pulls, and runs containers for you.

IMPORTANT NOTES

- ✓ The daemon can manage many images and containers.
- ✓ Images are templates. Containers are running instances.
- ✓ Registries store images so they can be shared and retrieved.



INSTALLING DOCKER AND VERIFYING THE SETUP

DOCKER DESKTOP VS DOCKER ENGINE

DOCKER DESKTOP



- Easy to install
- GUI included
- Best for beginners
- Runs a lightweight VM behind the scenes

DOCKER ENGINE



- Server/CLI only
- Lightweight
- Best for servers and production
- Runs natively on the host OS

PLATFORM OVERVIEW



LINUX

Install Docker Engine using package manager.
Works on most distributions.



MACOS

Install Docker Desktop for Mac.
Uses a lightweight Linux VM.



WINDOWS

Install Docker Desktop for Windows.
Uses a lightweight Linux VM.

1 CHECK DOCKER VERSION

```
docker --version
```

Shows the installed Docker version.



2 CHECK DOCKER INFO

```
docker info
```

Displays detailed information about the Docker installation, server, storage, and system.



3 RUN HELLO-WORLD

```
docker run hello-world
```

Downloads a test image (if needed) and runs a container successfully.



WHAT HAPPENS DURING THE FIRST CONTAINER RUN?



1 IMAGE NOT FOUND LOCALLY

Docker checks locally for the hello-world image.



2 IMAGE PULLED FROM DOCKER HUB

If not found, Docker pulls the image from Docker Hub.



3 CONTAINER CREATED & STARTED

Docker creates a new container from the image and starts it.



4 MESSAGE DISPLAYED

The container runs a simple test and prints a success message.



5 CONTAINER STOPS AUTOMATICALLY

The container exits after completing the test.

COMMON INSTALLATION PROBLEMS



DOCKER SERVICE NOT STARTING

- Start Docker service.
- Check system logs.
- Restart your machine.



PORT CONFLICTS

- Docker needs certain ports.
- Stop conflicting services.
- Change the port if required.



VIRTUALIZATION DISABLED

- Enable VT-x/AMD-V in BIOS.
- Required for Docker Desktop.



IMAGE PULL FAILURES

- Check internet connection.
- Check DNS settings.
- Try again or change registry.



PERMISSION DENIED (LINUX)

- Add user to docker group.
 - Use sudo if needed.
- `sudo usermod -aG docker $USER`



STORAGE / DISK SPACE ISSUES

- Docker images use disk space.
 - Clean unused images/containers.
- `docker system prune -a`



TIP: After installation, run the three commands below to verify everything is working correctly.

```
docker --version
```

```
docker info
```

```
docker run hello-world
```

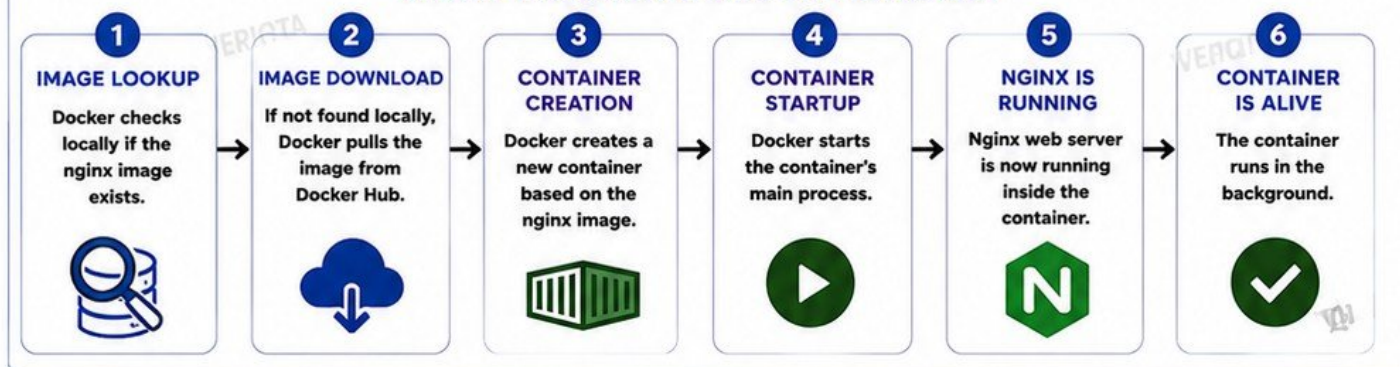


YOUR FIRST REAL CONTAINER

>_ docker run nginx

This command downloads the Nginx image (if needed) and runs a container from it.

WHAT DOCKER DOES INTERNALLY



CHECK RUNNING CONTAINERS

>_ docker ps

Shows only currently running containers.

CONTAINER ID	IMAGE	COMMAND	STATUS	PORTS	NAMES
f3e5c1a2b7d9	nginx	"nginx -g 'daemon..."	Up 5s	80/tcp	serene_wiles

CHECK ALL CONTAINERS

>_ docker ps -a

Shows all containers (running and stopped).

CONTAINER ID	IMAGE	COMMAND	STATUS	PORTS	NAMES
f3e5c1a2b7d9	nginx	"nginx -g 'daemon..."	Up 5s	80/tcp	serene_wiles
a1b2c3d4e5f6	nginx	"nginx -g 'daemon..."	Exited (0) 2 mins ago		goofy_carson



UNDERSTANDING CONTAINER IDs

- The **CONTAINER ID** is a unique identifier assigned to each container.
- It is long and random.
- You can use the first 12 characters (e.g., f3e5c1a2b7d9) to reference it.

EXAMPLE CONTAINER ID → f3e5c1a2b7d9e8c6a4d2f1b9a7c3d5e6f8g9h0i
 First 12 Characters
 f3e5c1a2b7d9



UNDERSTANDING CONTAINER NAMES

- Every container gets a human-readable name.
- If you don't specify a name, Docker generates one randomly (e.g., serene_wiles).
- You can give your container a custom name using **--name**.

EXAMPLE CUSTOM NAME → **docker run --name my-nginx nginx**
 Now the container name will be: my-nginx



QUICK SUMMARY

- You run: **docker run nginx**
- Docker looks for the image locally.
- If missing, it pulls the image from Docker Hub.
- Docker creates and starts a container.
- Nginx is running inside the container.
- You can manage it using docker commands.
- You can list containers using **docker ps** and **docker ps -a**.



DEFAULT PORT MAPPING NOTE

- By default, the Nginx container exposes port 80 inside the container.
- To access it from your browser, run:

docker run -p 8080:80 nginx

Then open: **http://localhost:8080**

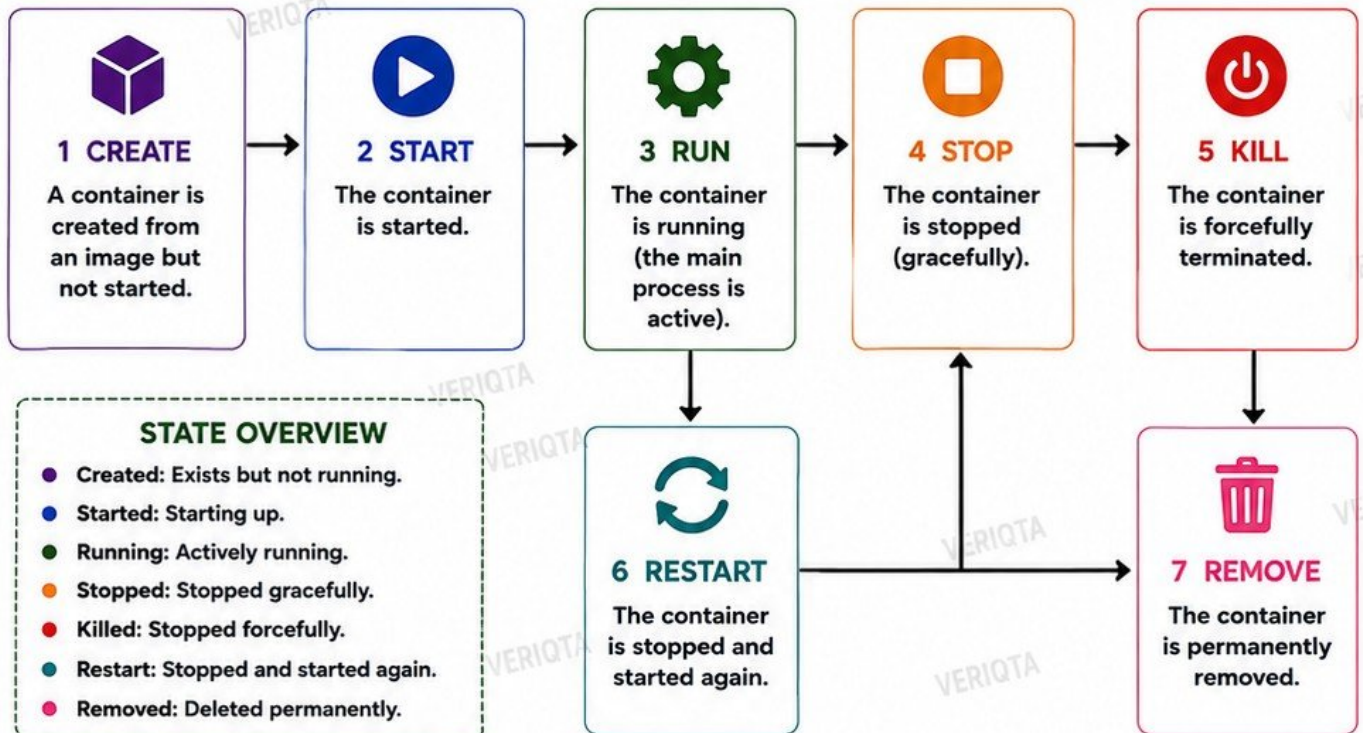


Without **-p**, the container runs but is not accessible from your browser.



CONTAINER LIFECYCLE

EVERY CONTAINER GOES THROUGH THESE STATES



DOCKER COMMANDS FOR EACH ACTION



EXAMPLE LIFECYCLE FLOW



```

docker create nginx
docker start <id_or_name>
docker stop <id_or_name>
  
```

```

docker restart <id_or_name>
docker rm <id_or_name>
  
```

IMPORTANT NOTES

- ★ You cannot start, stop, or restart a container that does not exist.
- ★ You cannot remove a container that is running. Stop it first.
- ★ Use `docker ps -a` to see all containers (including stopped ones).
- ★ Use `docker rm -f` to force remove a container (stops and removes).



WORKING INSIDE CONTAINERS

1. WHY WE ACCESS A CONTAINER

- Debug running applications
- Inspect files and directories
- Check logs and configurations
- Verify processes and services
- Run one-time admin/diagnostic commands



RUNNING CONTAINER

2. DOCKER EXEC BASICS

docker exec lets you run a command inside a running container.

```
docker exec <container_name_or_id> <command>
```

Examples:

```
docker exec web-container ls /
docker exec web-container pwd
docker exec web-container cat /etc/os-release
```

3. INTERACTIVE TERMINALS

To get an interactive shell inside the container, use the `-it` flags.

```
docker exec -it <container_name_or_id> <shell>
```

Example:

```
docker exec -it web-container /bin/bash
# or
docker exec -it web-container /bin/sh
```

4. /BIN/BASH VS /BIN/SH

SHELL	BASED ON	WHEN TO USE
<code>/bin/bash</code>	Bash (full features)	When available. Better experience and features.
<code>/bin/sh</code>	POSIX shell (minimal)	Always available in most images. Use when bash is not installed.

5. INSPECTING FILES AND DIRECTORIES



- List files: `ls -lah`
- Change directory: `cd /path`
- View file: `cat /path/file`
- Search file: `grep "text" /path/file`
- Show disk usage: `df -h`

6. CHECKING PROCESSES



- List processes: `ps aux`
- Show in real-time: `top`
- Filter process: `ps aux | grep nginx`
- Check listening ports: `ss -tln`

7. EXITING WITHOUT STOPPING THE CONTAINER



- Type: `exit` (clean way)
- Or press: `Ctrl + D` (end of input)
- This closes your shell session
- The container keeps running

8. WHY CONTAINERS ARE NOT SERVERS



- Containers are **EPHEMERAL** (replaceable)
- Changes inside can be **LOST** when the container is recreated
- **Do NOT** install software or make manual changes inside running containers
- Keep containers **IMMUTABLE**
- Build a new image and redeploy instead

9. QUICK REFERENCE COMMANDS

```
docker exec -it <ctr> /bin/bash
docker exec -it <ctr> /bin/sh
docker exec <ctr> ls /
docker exec <ctr> cat /etc/os-release
docker exec <ctr> ps aux
docker exec <ctr> exit
```

```
# Open bash shell inside container
# Open sh shell (fallback)
# List root directory
# Show OS information
# Show running processes
# Exit shell (container keeps running)
```



REMEMBER: Access containers for debugging and inspection only. Do not treat them like traditional servers.





UNDERSTANDING DOCKER IMAGES

1. WHAT AN IMAGE CONTAINS



- A blueprint for containers
- Application code
- Runtime environment
- System libraries
- Dependencies
- Metadata (author, layers, config)
- Default command (CMD/ENTRYPOINT)

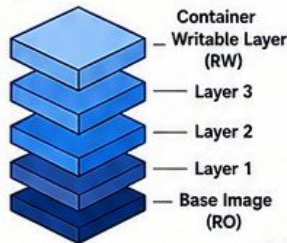


Images are templates.
Containers are running instances.

2. IMAGE VS CONTAINER

IMAGE	CONTAINER
Read-only	Read-write layer
Immutable	Mutable
Template/Blueprint	Running instance
Stored on disk	Runs in memory
Built or pulled	Created from an image
Used to create containers	Can be started, stopped, removed

3. IMAGE LAYERS



- Images are built in layers.
- Each instruction in a Dockerfile creates a new layer.
- Layers are stacked on top of each other.
- Layers are cached and reused.
- Only the top writable layer changes in a container.



Layered design makes images efficient, fast to build, and easy to share.

4. READ-ONLY IMAGE CONCEPT



- Images are read-only.
- You cannot change files inside an image.
- When a container runs, Docker adds a thin writable layer on top.
- Changes inside the container exist only until the container is removed.

IMAGE (READ-ONLY)



RUN

CONTAINER



Writable Layer (RW)

5. TAGS



A tag is a version label for an image.

- Format: name:tag
- If no tag is specified, Docker uses: **latest**

Examples:

```
nginx:latest
nginx:1.25
ubuntu:22.04
redis:7-alpine
```



Always use specific tags in production.

6. LIST IMAGES



List all images on your system.

```
docker images
```

List with more details:

```
docker images -a
```

REPOSITORY	TAG	IMAGE ID
nginx	latest	a1b2c3d4e5f6
redis	7	b1c2d3e4f5g6
ubuntu	22.04	c1d2e3f4a5b6



Shows local images stored on your system.

7. PULL AN IMAGE



Download an image from a registry (Docker Hub by default).

```
docker pull nginx:1.25
```

Pull without tag (pulls latest):

```
docker pull ubuntu
```



Downloads the image layers if not present locally.

8. REMOVE AN IMAGE



Remove an image from your system.

```
docker rmi nginx:1.25
```

Force remove (if needed):

```
docker rmi -f nginx:1.25
```



Cannot remove an image that is being used by running containers.

9. WHY IMAGE SIZE MATTERS



FASTER PULLS
Smaller images download quicker.



LESS BANDWIDTH
Saves network and data costs.



LESS STORAGE
Uses less disk space on hosts and CI/CD runners.



BETTER SECURITY
Smaller images have fewer packages, less attack surface.



FASTER DEPLOYMENTS
Quicker startups and scaling.



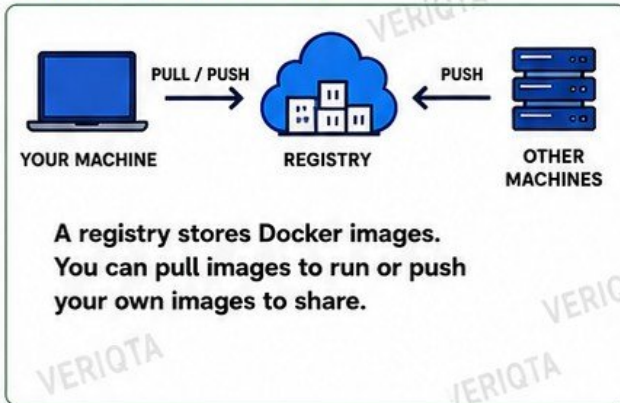
TIP: Use minimal base images (alpine, distroless), clean up unnecessary files, and combine Dockerfile layers to keep images small.





DOCKER HUB AND CONTAINER REGISTRIES

1. WHAT A REGISTRY DOES



2. REPOSITORY vs IMAGE vs TAG

REPOSITORY	IMAGE	TAG
 A collection of related images. (Like a project folder) Example: <code>nginx</code>	 A specific version of the software inside a repository. Example: <code>nginx</code>	 A label for an image version inside a repository. Example: <code>nginx:1.25</code>
Full reference: <code>docker.io/library/nginx:1.25</code>		

3. DOCKER HUB

- Docker Hub is the default public registry.
- Millions of images available.
- Easy to search, pull and use.
- You can also push your own images.

Default public registry: `docker.io`

4. OFFICIAL IMAGES

- Created and maintained by Docker or trusted partners.
- High quality, regularly updated.
- Follows best practices.
- Look for the "OFFICIAL IMAGE" badge on Docker Hub.

5. PULLING IMAGES

Download an image from a registry to your local machine.

```
docker pull nginx:1.25
# or
docker pull ubuntu
```

If no tag is specified, Docker pulls the "latest" tag by default.

6. IMAGE TAGS

- Tags identify specific versions.
- Examples: `1.25`, `1.24`, `22.04`, `alpine`
- You can create your own tags.
- Always use meaningful tags in production.

```
docker pull nginx:1.25
docker pull nginx:alpine
docker pull nginx:1.25.3
```

7. WHAT IS "LATEST"?

- "latest" is just a moving label.
- It points to the most recently pushed image.
- It can change at any time.
- Not a specific version!

8. WHY TEAMS AVOID BLINDLY USING "LATEST"

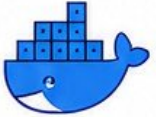
- Can change without notice.
- May introduce breaking changes.
- Builds may break unexpectedly.
- Harder to reproduce and debug issues.
- Not reliable for CI/CD and production.

Best Practice: Use specific, versioned tags (e.g., `1.25.3`) in production.

9. BASIC IMAGE TRUST & SECURITY

 USE OFFICIAL IMAGES Prefer official or trusted publisher images.	 CHECK IMAGE SOURCE Verify the publisher and repository before pulling.	 SCAN IMAGES Scan for vulnerabilities using tools like Docker Scout, Trivy, or Gype.	 KEEP IMAGES UPDATED Use updated tags that include security patches.	 MINIMIZE BASE IMAGES Use small, minimal base images (alpine, distroless) to reduce attack surface.
--	--	---	---	--

★ Pull only what you trust. Secure images = Secure containers = Secure applications.



YOUR FIRST DOCKERFILE

BUILD YOUR OWN IMAGE FROM SCRATCH

1. WHAT IS A DOCKERFILE?



- A Dockerfile is a text file that contains instructions to build a Docker image.
- Each instruction creates a layer in the image.
- You build images, then run containers from those images.

2. DOCKERFILE KEY INSTRUCTIONS

INSTRUCTION	PURPOSE
FROM	Sets the base image.
WORKDIR	Sets the working directory inside the container.
COPY	Copies files from your machine into the image.
RUN	Runs commands during image build.
CMD	Sets the default command when the container starts.

3. EXAMPLE: SIMPLE NODE.JS APP

my-app/

├── Dockerfile
├── package.json
└── app.js

app.js (example)

```
const http = require('http');
const port = 3000;

http.createServer((req, res) => {
  res.end('Hello from Docker!');
}).listen(port, () => {
  console.log(`Server running on port ${port}`);
});
```

4. DOCKERFILE EXAMPLE

```
# Use official Node.js image as base
FROM node:20-alpine

# Set working directory inside container
WORKDIR /app

# Copy files into the container
COPY package*.json ./

# Install dependencies
RUN npm install

# Copy the rest of the application code
COPY . .

# Expose application port
EXPOSE 3000

# Default command to run the app
CMD ["node", "app.js"]
```

5. BUILD YOUR IMAGE

Build the image from the directory containing the Dockerfile.

```
# Build with default tag (optional)
docker build .
```

```
# Build with a custom name and tag
docker build -t my-app:1.0 .
```



-t means "tag". It gives your image a name and version.

6. RUN YOUR CONTAINER

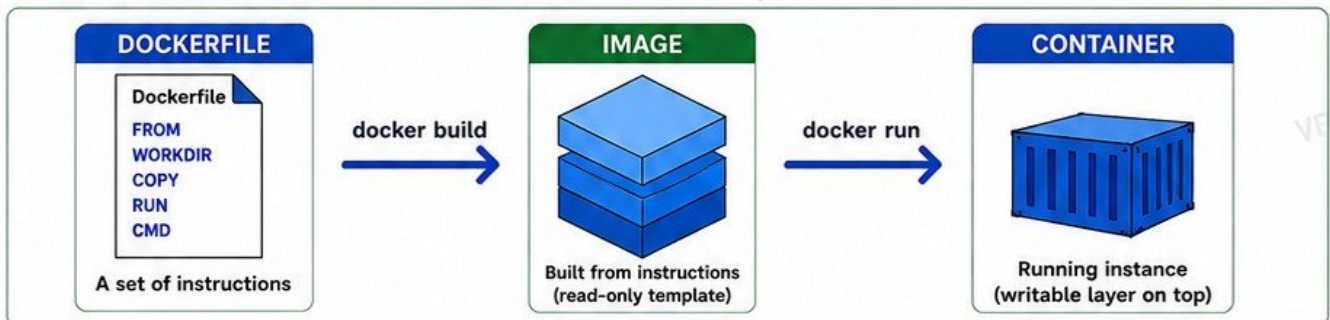
```
docker run -d -p 3000:3000 my-app:1.0
```

-d : Run in detached mode (background)

-p 3000:3000 : Map host port 3000 to container port 3000

my-app:1.0 : The image name and tag

7. DOCKERFILE → IMAGE → CONTAINER



You write a Dockerfile, build an image, and run containers from that image!



UNDERSTANDING DOCKERFILE INSTRUCTIONS

WHAT EACH INSTRUCTION DOES AND WHEN TO USE IT

INSTRUCTION	PURPOSE	WHEN TO USE	EXAMPLE
 FROM	Sets the base image for your build.	Always the first instruction. Defines the starting point.	<code>FROM ubuntu:22.04</code>
 RUN	Runs commands in the image and creates a new layer.	Install packages, update system, configure tools.	<code>RUN apt-get update && apt-get install -y curl</code>
 COPY	Copies files or directories from your machine to the image.	Use for local files and folders you want inside the image.	<code>COPY . /app</code>
 ADD	Copies files and also supports URLs and archives.	Use when you need to download or auto-extract tar files.	<code>ADD https://example.com/app.tar.gz /app</code>
 WORKDIR	Sets the working directory for any subsequent instruction.	To avoid using long paths in RUN, CMD, ENTRYPOINT, etc.	<code>WORKDIR /app</code>
 ENV	Sets environment variables in the image.	For app configuration, paths, timezones, etc.	<code>ENV NODE_ENV=production</code>
 EXPOSE	Documents the port the container listens on.	To inform users which port your app uses.	<code>EXPOSE 8080</code>
 USER	Sets the user that runs the container.	To improve security. Avoid running as root.	<code>USER appuser</code>
 CMD	Default command to run when container starts.	To define the default behavior of the container.	<code>CMD ["node", "server.js"]</code>
 ENTRYPOINT	Configures a container that will run as an executable.	For containers that should always run a specific command.	<code>ENTRYPOINT ["nginx"]</code>

COMMON BEGINNER MISTAKES

- ❌ **Installing unnecessary packages**
Makes images bigger and slower.
- ❌ **Not using .dockerignore**
Adds unnecessary files to the build context.
- ❌ **Using too many RUN layers**
Creates more layers and larger images.
- ❌ **Forgetting to clean up in RUN**
Use: `apt-get clean && rm -rf /var/lib/apt/lists/*`
- ❌ **Not using specific base image tags**
Avoid "latest". Use specific version tags.
- ❌ **Running as root in production**
Always use a non-root user.
- ❌ **Confusing CMD and ENTRYPOINT**
Use CMD for defaults, ENTRYPOINT for fixed commands.

DOCKERFILE EXAMPLE (SIMPLE NODE APP)

```
1 FROM node:20-alpine      # Base image
2 WORKDIR /app              # Set working directory
3 COPY package*.json ./     # Copy dependency files
4 RUN npm install            # Install dependencies
5 COPY . .                  # Copy application code
6 ENV NODE_ENV=production   # Set environment variable
7 EXPOSE 3000               # Document application port
8 USER node                 # Run as non-root user
9 CMD ["node", "server.js"] # Default command
10
```



**BUILD
AND RUN**

```
docker build -t my-node-app .
docker run -p 3000:3000 my-node-app
```




PORTS AND CONTAINER NETWORKING

MAKE YOUR CONTAINER REACHABLE

1. WHY A RUNNING CONTAINER MAY STILL BE UNREACHABLE



- Containers have their own network.
- By default, ports are NOT exposed to the host.
- Other computers (or browsers) cannot reach the container.
- You must publish (map) ports to make them reachable.

2. CONTAINER PORTS VS HOST PORTS

CONTAINER PORT	HOST PORT
 The port inside the container where the app is listening.	 The port on the host machine that forwards traffic to the container port.

3. EXPOSE VS PUBLISHED PORTS

EXPOSE



- Declares the port the container listens on.
- Documentation only.
- Does NOT publish the port to the host.

PUBLISHED PORTS



- Uses `-p` or `-P` in docker run.
- Maps a host port to the container port.
- Makes the container reachable from outside.

4. PUBLISH PORTS WITH DOCKER RUN -P

```
>_ docker run -p <host_port>:<container_port> <image>
```

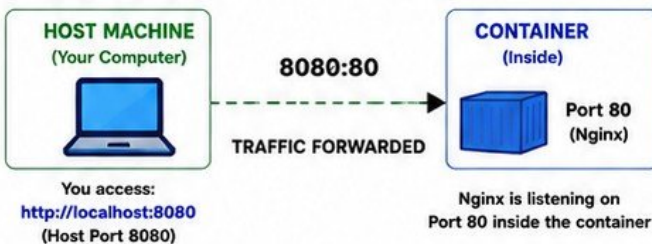
Examples:

```
$ docker run -p 8080:80 nginx
$ docker run -p 5000:5000 myapp
$ docker run -p 3000:3000 node-app
```

8080:80

8080 = Host Port (outside the container)
80 = Container Port (inside the container)

5. 8080:80 EXPLAINED VISUALLY



6. CHECK PUBLISHED PORTS

```
>_ docker port <container_name_or_id>
```

Example:

```
$ docker port nginx-container
80/tcp -> 0.0.0.0:8080
```

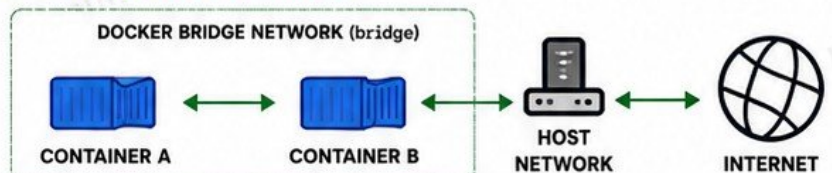
Container Port Host IP Host Port

7. BASIC BRIDGE NETWORKING

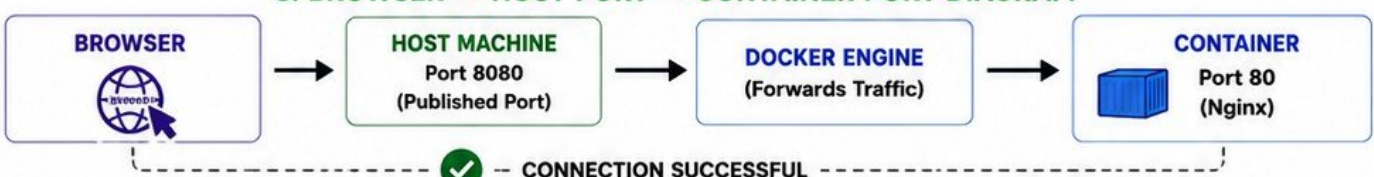
WHAT IS BRIDGE NETWORK?



- Default network driver in Docker.
- Containers on the same bridge network can talk to each other.
- Containers can reach the outside world through the host.



8. BROWSER → HOST PORT → CONTAINER PORT DIAGRAM





DOCKER NETWORKS

CONNECT CONTAINERS. COMMUNICATE. SCALE.

1. DEFAULT BRIDGE NETWORK



bridge

- Created automatically by Docker.
- Used when you don't specify a network.
- Containers can communicate with each other on the same bridge.
- Not recommended for production multi-container apps.
- Name: `bridge`

DEFAULT BRIDGE NETWORK



2. USER-DEFINED BRIDGE NETWORKS



- You create the network.
- Better isolation and control.
- Automatic DNS between containers.
- Easier management.
- Recommended for multi-container apps.

USER-DEFINED BRIDGE NETWORK (example: app-net)



3. LIST NETWORKS

```
> docker network ls
```

Example Output:

NETWORK ID	NAME	DRIVER	SCOPE
c4f2a1b7e9c6	bridge	bridge	local
89d2c6a5f7b1	host	host	local
d3e7a6b2c4f5	none	null	local
f6a7b8c9d8e1	app-net	bridge	local

4. CREATE NETWORK

```
> docker network create app-net
```

Example Output:

```
f6a7b8c9d8e1b3c4d5e6f7a8b9c0d1e2f3a4b5c
```

5. INSPECT NETWORK

```
> docker network inspect app-net
```

Example Output (short):

```
{
  "Name": "app-net",
  "Driver": "bridge",
  "Scope": "local",
  "Containers": {...}
}
```

6. CONNECTING CONTAINERS TO A NETWORK



Option 1: Connect at run time

```
> docker run -d --name api --network app-net myapi
```



Option 2: Connect an existing container

```
> docker network connect app-net existing-container
```



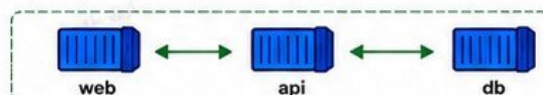
Disconnect a container

```
> docker network disconnect app-net container-name
```

7. CONTAINER-NAME DNS



- Containers on the same user-defined network can reach each other by name.
- Docker provides automatic DNS resolution.
- Example: api can reach db using "db".



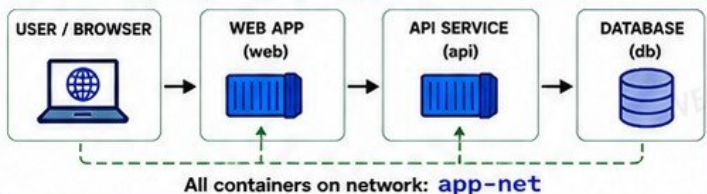
api connects to db using the name "db", not an IP address.

8. WHY YOU SHOULD NOT HARD-CODE CONTAINER IP ADDRESSES



- Container IPs change when containers restart.
- IP addresses can change across environments.
- Services can scale (more containers = more IPs).
- Hard-coding breaks communication.
- Use container names + user-defined networks for reliability.

9. APP → API → DATABASE EXAMPLE



10. QUICK COMMAND SUMMARY



List networks
`docker network ls`



Create network
`docker network create <name>`



Inspect network
`docker network inspect <name>`



Connect container
`docker network connect <name> <container>`



Disconnect container
`docker network disconnect <name> <container>`

REMEMBER:

Use user-defined networks, container names, and service discovery.
Avoid hard-coded IP addresses for scalable and reliable applications.





DOCKER STORAGE AND VOLUMES

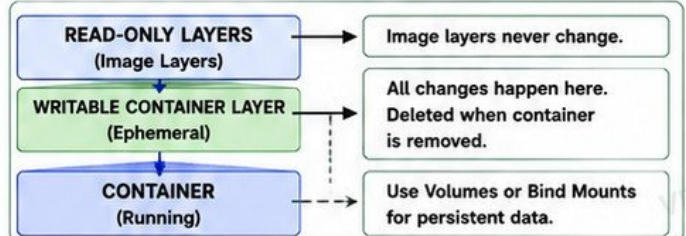
PERSISTENT DATA FOR CONTAINERS

1. WHY CONTAINER DATA DISAPPEARS



- Container writable layer is ephemeral.
- When a container is removed, all changes inside are lost.
- Databases, uploads, and files will be deleted.
- Use Volumes or Bind Mounts to persist data.

2. CONTAINER FILE SYSTEM LAYERS



3. VOLUMES (MANAGED BY DOCKER)



- Stored in Docker's storage area.
- Managed by Docker.
- Portable across containers.
- Better for databases and critical data.
- Survive container removal.

4. BIND MOUNTS (MANAGED BY HOST)



- Map a host directory or file into container.
- Full control from the host.
- Host path must exist.
- Good for source code, logs, configs.
- Changes reflect immediately.

5. CREATE VOLUME

```
> docker volume create mydata
```

Example Output:

```
1f2d3e4c5b6a7d8e9f8a1234567890abcdef1234567890abcdef12345
```

6. LIST VOLUMES

```
> docker volume ls
```

Example Output:

```
DRIVER    VOLUME NAME
local     mydata
local     db-data
```

7. INSPECT VOLUME

```
> docker volume inspect mydata
```

Example Output (short):

```
{
  "Name": "mydata",
  "Driver": "local",
  "Mountpoint": "/var/lib/docker/volumes/mydata/_data"
}
```

8. -v AND --mount (TWO WAYS TO MOUNT)

-v (Short Syntax)

```
> docker run -d --name web -v mydata:/var/www/html nginx
```

Volume Name Container Path Image

--mount (Long Syntax - More Powerful)

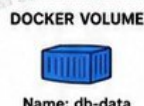
```
> docker run -d --name db --mount type=volume,source=db-data,target=/var/lib/mysql mysql
```

Type Source Target Image
(volume/bind/tmpfs) (volume name or host path) (container path)

9. COMMON MOUNT EXAMPLES

USE CASE	COMMAND EXAMPLE	DESCRIPTION
Named Volume	<code>-v mydata:/data</code>	Docker-managed volume → /data
Bind Mount	<code>-v /host/path:/app</code>	Host folder → Container folder
Read-Only Mount	<code>-v /host/config:/app/config:ro</code>	Mount as read-only
Long Syntax	<code>--mount type=volume,source=mydata,target=/data</code>	More options and clarity

10. PERSISTENT DATABASE EXAMPLE (MYSQL)



Steps:

1 Create volume

```
docker volume create db-data
```

2 Run MySQL container

```
docker run -d --name mysql \
-v db-data:/var/lib/mysql \
-e MYSQL_ROOT_PASSWORD=secret \
mysql:8
```

3 Stop and remove container

```
docker stop mysql
docker rm mysql
```

4 Run container again

```
docker run -d --name mysql \
-v db-data:/var/lib/mysql \
-e MYSQL_ROOT_PASSWORD=secret \
mysql:8
```

5 Data is still there!

Database files are safe in the volume.



KEY TAKEAWAY

Always use Volumes or Bind Mounts for data that must survive container restarts, removals, or recreations.



BEST PRACTICES

- Use named volumes for databases.
- Use bind mounts for development (code, logs).
- Never store critical data only in the container writable layer.



ENVIRONMENT VARIABLES AND CONFIGURATION

CONFIGURE SMARTER. DEPLOY SAFER.

1. WHY CONFIGURATION SHOULD NOT BE BAKED INTO IMAGES



- Images should be portable and reusable.
- Different environments need different values.
- Hard-coded values require image rebuilds.
- Increases risk of leaking secrets.
- Breaks 12-Factor App principles.



Use environment variables, files, and secrets management instead.

2. PASS ENVIRONMENT VARIABLES WITH docker run -e

```
>_ docker run -e KEY=VALUE -e KEY2=VALUE2 image
```

Example:

```
$ docker run -e NODE_ENV=production \  
-e PORT=3000 \  
-e DB_HOST=db \  
myapp
```



Inside the container, the application can access these using `process.env.KEY` (Node.js) or `os.getenv("KEY")` (Python, Java, etc).

3. ENVIRONMENT FILES (USE --ENV-FILE)

```
>_ docker run --env-file .env myapp
```

Example .env file

```
NODE_ENV=production  
PORT=3000  
DB_HOST=db  
DB_USER=appuser  
DB_PASS=secret123
```

- Store multiple variables in a file.
- Cleaner and easier to manage.
- **Do NOT** commit sensitive .env files to version control.

4. APPLICATION CONFIGURATION



Applications read configuration from:

- Environment variables
- Config files
- Command-line args
- External services (e.g. config servers)



GOOD PRACTICE

Keep code the same. Change configuration per environment.

5. DEVELOPMENT VS PRODUCTION CONFIGURATION



DEVELOPMENT

- Debug logs enabled
- Hot reload
- Local database
- Less strict security
- More verbose errors

PRODUCTION

- Minimal logging
- Optimized performance
- Managed database
- Strict security
- Clean error messages



TIP: Use environment variables to control these differences without changing the image.

6. BASIC SECRET-HANDLING WARNING



NEVER store secrets in:

- Dockerfiles
- Source code
- Image layers
- Version control



Instead, use:

- Docker secrets (Swarm)
- External secret managers (AWS Secrets Manager, HashiCorp Vault, Azure Key Vault, etc.)
- Environment variables injected at runtime
- Kubernetes secrets (when using Kubernetes)

7. WHY PASSWORDS SHOULD NOT BE HARD-CODED INTO DOCKERFILES

DOCKERFILE (BAD EXAMPLE)

```
FROM mysql:8  
ENV MYSQL_ROOT_PASSWORD=MySecret123  
ENV DB_PASSWORD=MySecret123
```

PROBLEMS

- Visible in image history
- Stored in image layers forever
- Anyone with image can extract
- High security risk
- Violates security best practices

BETTER APPROACH

```
docker run -e MYSQL_ROOT_PASSWORD=...  
or  
docker run --env-file .env mysql:8  
or  
Use Docker/K8s secrets
```



Keep secrets out of images. Keep apps secure.



QUICK COMMAND REFERENCE

```
docker run -e KEY=VALUE  
Set environment variable
```

```
docker run --env-file .env  
Use environment file
```

```
docker exec -it container env  
View variables inside container
```

```
docker inspect container  
See environment variables
```



REMEMBER:

Keep configuration flexible. Keep secrets safe. Build once, run anywhere, securely.



DOCKER LOGS, INSPECT AND TROUBLESHOOTING

FIND PROBLEMS. UNDERSTAND ROOT CAUSE. FIX FAST.

1. ESSENTIAL DEBUGGING COMMANDS

<code>>_ docker logs <container></code>	Show logs from a container.
<code>>_ docker logs -f <container></code>	Stream logs in real time (follow mode).
<code>>_ docker inspect <container></code>	Detailed JSON information about container configuration and state.
<code>>_ docker stats <container></code>	Live resource usage (CPU, MEM, NET, I/O).
<code>>_ docker top <container></code>	Show running processes inside the container.

2. CHECKING EXIT CODES

```
>_ docker inspect -f '{{.State.ExitCode}}' <container>
```

Example:

```
$ docker inspect -f '{{.State.ExitCode}}' myapp
1
```

Exit codes:

- 0 = Successful
- 1 = General error
- 125 = Docker CLI or daemon error
- 126 = Command invoked cannot execute
- 127 = Command not found
- 128+ = Error signal



Always check exit code + logs to understand why a container stopped.

3. COMMON CONTAINER PROBLEMS

 CONTAINER STARTS THEN IMMEDIATELY EXITS	- Application crashed. - Wrong command or CMD. - Missing dependencies. - Check logs and exit code.
 PORT CONFLICTS	- Host port already in use. - Check with: <code>netstat -tulin</code> or <code>ss -tulin</code> - Use a different host port.
 MISSING ENVIRONMENT VARIABLES	- Application needs a value. - Check with <code>docker inspect</code>. - Use <code>-e</code> or <code>--env-file</code>.
 PERMISSION PROBLEMS	- File permission denied. - Running as root vs non-root. - Fix with proper user or <code>chmod</code>.
 CONTAINER NOT STARTING AT ALL	- Wrong image or command. - Dependency issues. - Check logs and error message.

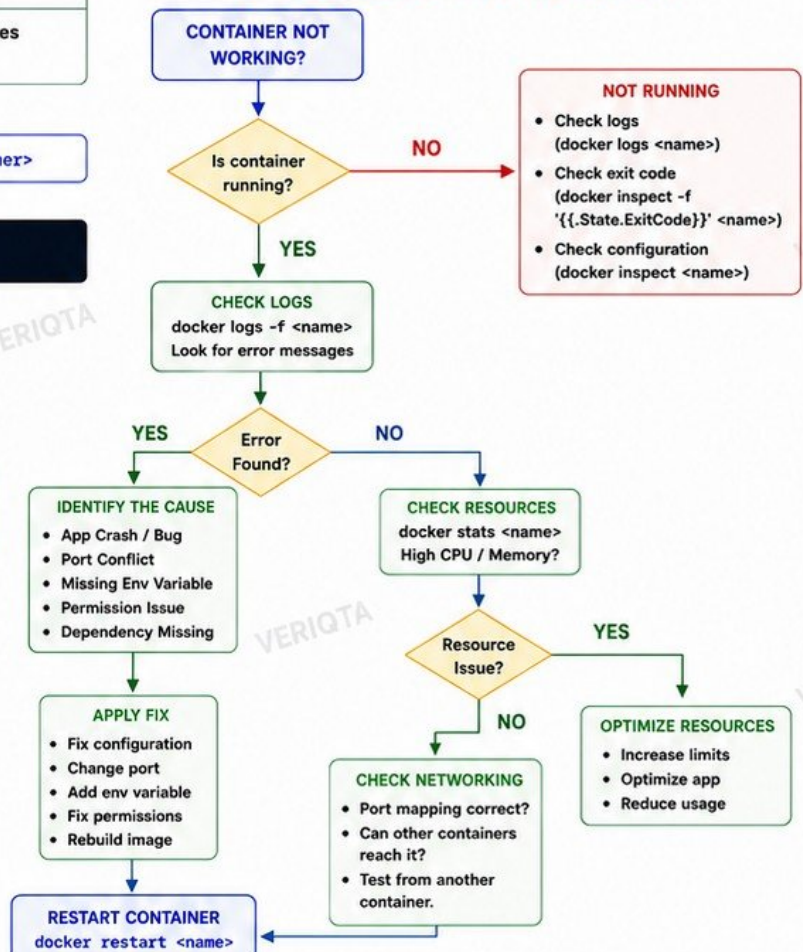
6. PRO TIPS

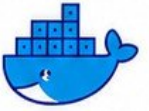
 CHECK LOGS FIRST Logs tell you what is really happening.	 USE INSPECT WISELY It reveals everything: env, mounts, ports, networks, status.	 DON'T GUESS Use commands. Understand the root cause.	 KEEP IT SIMPLE One change at a time. Restart and test.	 BUILD RESILIENT APPS Handle errors, log properly, and exit with correct codes.
--	---	--	---	--

4. QUICK EXAMPLES

<code>docker logs myapp</code>	View logs from container.
<code>docker logs -f myapp</code>	Follow logs in real time.
<code>docker inspect myapp</code>	Full details about container.
<code>docker stats myapp</code>	Check live resource usage.
<code>docker top myapp</code>	See running processes.
<code>docker inspect -f '{{.State.ExitCode}}' myapp</code>	Get exit code.

5. SIMPLE TROUBLESHOOTING FLOWCHART





INTRODUCTION TO DOCKER COMPOSE

RUN MULTI-CONTAINER APPLICATIONS WITH A SINGLE FILE AND SIMPLE COMMANDS

1. WHY MULTIPLE DOCKER RUN COMMANDS BECOME DIFFICULT

- ✗ Too many commands to remember and maintain
- ✗ Hard to manage dependencies between containers
- ✗ Manual network creation and connection
- ✗ Port mapping becomes messy and error-prone
- ✗ Environment variables are hard to organize
- ✗ Difficult to share setup with your team
- ✗ Not reproducible – easy to miss a step
- ✗ Scaling or recreating the stack is tedious

2. WHAT DOCKER COMPOSE SOLVES

- ✓ Define your entire application in one compose.yaml file
- ✓ Start all services with a single command
- ✓ Automatically create and connect networks
- ✓ Manage environment variables cleanly
- ✓ Manage volumes for persistent data
- ✓ Easy to share and version control
- ✓ Reproducible across environments
- ✓ Stop and clean everything with one command

3. COMPOSE.YAML – THE HEART OF COMPOSE

A simple YAML file that tells Docker Compose how your application should run.



compose.yaml

- Services
- Images / Builds
- Ports
- Environment Variables
- Volumes
- Networks
- Dependencies



4. KEY CONCEPTS IN COMPOSE

	SERVICES	A service represents a container in your application.
	IMAGES	Use existing images from Docker Hub or your own.
	BUILDS	Build your own image from a Dockerfile.
	PORTS	Map container ports to host ports.
	ENVIRONMENT VARIABLES	Pass configuration and secrets (non-sensitive).
	VOLUMES	Persist data outside the container.
	NETWORKS	Connect services so they can communicate with each other.

5. EXAMPLE COMPOSE.YAML (SIMPLE WEB APP)

```
version: '3.9'
services:
  web:
    build: .           # Build from Dockerfile in current dir
    ports:
      - "8080:80"      # Host:Container
    environment:
      - NODE_ENV=production # Environment variable
    volumes:
      - ./:/app         # Bind mount
    depends_on:
      - db              # Start db before web
  db:
    image: postgres:15 # Use image from Docker Hub
    environment:
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=pass
      - POSTGRES_DB=myapp # DB credentials
    volumes:
      - db_data:/var/lib/postgresql/data # Named volume
volumes:
  db_data: # Declare named volume
networks:
  default: # Default network
```

6. START YOUR STACK



docker compose up

- Reads compose.yaml
- Creates networks, volumes, and containers
- Starts all services
- Shows logs in the terminal
- Use -d to run in detached mode (background)

`docker compose up -d`

7. STOP AND CLEAN UP



docker compose down

- Stops all running containers
- Removes containers, networks, and (default) volumes
- Leaves your compose.yaml file intact

`docker compose down -v`



COMPOSE BENEFITS:



SAVES TIME



EASY FOR TEAMS



REPRODUCIBLE



REDUCES ERRORS



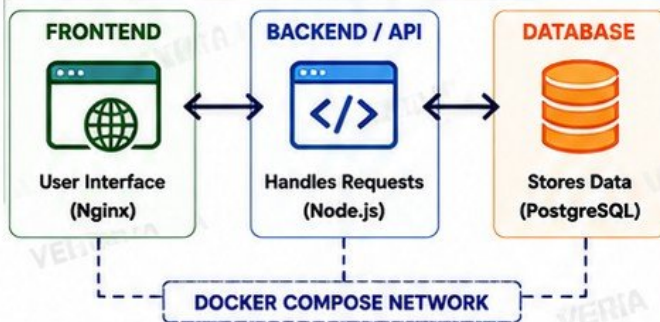
SCALABLE



BUILD A MULTI-CONTAINER APPLICATION

BUILD, CONNECT, AND RUN YOUR COMPLETE STACK WITH DOCKER COMPOSE

1. THREE-TIER ARCHITECTURE DIAGRAM



2. WHAT WE WILL BUILD

- ✓ Frontend (Nginx) – Serves the web UI
- ✓ Backend/API (Node.js) – Handles API requests
- ✓ Database (PostgreSQL) – Stores application data
- ✓ All services connected on a custom Docker network
- ✓ Persistent volume for the database
- ✓ Environment variables for configuration
- ✓ Services communicate using service names
- ✓ Start, inspect, stop, and recreate the stack easily

3. DOCKER COMPOSE FILE (compose.yml)

```
version: '3.9'
services:
  frontend:
    image: nginx:alpine
    ports:
      - "8080:80"
    depends_on:
      - backend
    networks:
      - app-network
  backend:
    build: ./backend
    ports:
      - "5000:5000"
    environment:
      - DB_HOST=database
      - DB_USER=postgres
      - DB_PASSWORD=pass
      - DB_NAME=appdb
    depends_on:
      - database
    networks:
      - app-network
  database:
    image: postgres:15
    environment:
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=pass
      - POSTGRES_DB=appdb
    volumes:
      - db_data:/var/lib/postgresql/data
    networks:
      - app-network
volumes:
  db_data:
networks:
  app-network:
    driver: bridge
```

4. SERVICE-TO-SERVICE COMMUNICATION



5. START THE COMPLETE STACK



Start all services in the background.

```
docker compose up -d
```

6. INSPECT YOUR STACK



- View running containers: `docker compose ps`
- View logs (all services): `docker compose logs -f`
- Check a specific service: `docker compose logs -f backend`
- Inspect network: `docker network inspect <project>_app-network`
- Access frontend: `http://localhost:8080`
- Access API: `http://localhost:5000`
- Check database: `docker compose exec database psql -U postgres -d appdb`

7. STOP THE STACK



Stop and remove running containers, networks, and default volumes (data in named volumes is preserved).

```
docker compose down
```

8. RECREATE THE STACK

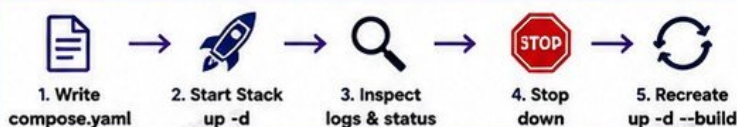


Rebuild images (if code changed) and start again.

```
docker compose down
```

```
docker compose up -d --build
```

QUICK WORKFLOW SUMMARY



BENEFITS

- ✓ Run entire applications with one file and one command
- ✓ Consistent environments for all team members
- ✓ Easy setup, teardown, and recreation
- ✓ Persistent data with named volumes
- ✓ Scalable and production-ready foundation



DOCKER BEST PRACTICES FOR BEGINNERS

BUILD SMALLER, SAFER, FASTER, AND PRODUCTION-READY CONTAINERS

1 USE SMALL BASE IMAGES APPROPRIATELY



- Smaller images pull faster and start faster.
- Use Alpine for small images where possible.
- Avoid heavy, unnecessary base images.

```
FROM node:20-alpine
```

2 PIN IMPORTANT IMAGE VERSIONS



- Never rely on "latest".
- Images can change without warning.
- Pin to a specific version or digest for stability.

```
FROM nginx:1.25.4-alpine
```

3 USE .DOCKERIGNORE



- Exclude unnecessary files from the build context.
- Reduces image size and build time.
- Prevents secrets and local files from leaking.

```
node_modules/
.git
.env
*.log
Dockerfile
```

4 REDUCE UNNECESSARY LAYERS



- Each RUN, COPY, ADD creates a layer.
- Combine commands when possible.
- Fewer layers = smaller and faster images.

```
RUN apt-get update && \
  apt-get install -y curl \
  && rm -rf /var/lib/apt/lists/*
```

5 DO NOT RUN APPLICATIONS AS ROOT WHEN UNNECESSARY



- Running as root is risky.
- Create a non-root user and use it.
- Improves container security.

```
RUN adduser -D appuser
USER appuser
```

6 DO NOT BAKE SECRETS INTO IMAGES



- Never put passwords, tokens, or keys in Dockerfile or code.
- Use environment variables, secrets managers, or runtime injection.

```
# BAD: ENV DB_PASS=supersecret
# GOOD: Pass secrets at runtime
```

7 KEEP CONTAINERS FOCUSED ON ONE PRIMARY RESPONSIBILITY



- One container = One job.
- Do not run multiple services in one container.
- Makes scaling, monitoring, and debugging easier.

```
Web | API | Worker | DB
(Each in its own container)
```

8 USE HEALTH CHECKS WHERE APPROPRIATE



- Health checks help Docker and orchestrators know if your app is healthy.
- Enables auto-restart and better reliability.

```
HEALTHCHECK --interval=30s \
  --timeout=5s --retries=3 \
  CMD curl -f http://localhost/health \
  || exit 1
```

9 REMOVE UNNECESSARY PACKAGES



- Do not install packages you do not need.
- Clean package caches and temp files.
- Keeps images small and secure.

```
RUN apt-get clean && \
  rm -rf /var/lib/apt/lists/* \
  /tmp/* /var/tmp/*
```

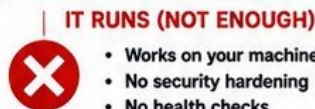
10 UNDERSTAND IMAGE VULNERABILITIES



- Base images may contain vulnerabilities.
- Scan images regularly.
- Keep base images updated.
- Use minimal and trusted images.

```
docker scan my-image
or integrate vulnerability scanning in CI/CD
```

11 "IT RUNS" VS "IT IS PRODUCTION-READY"



- IT RUNS (NOT ENOUGH)**
- Works on your machine
 - No security hardening
 - No health checks
 - Large image size
 - No monitoring
 - No backup plan



- PRODUCTION-READY**
- Secure by default
 - Non-root user
 - Small and scanned image
 - Health checks
 - Logs and monitoring
 - Backups and resilience
 - Configurable and reusable



- ✓ Build small. Build secure.
- ✓ Keep it simple and focused.
- ✓ Automate everything.
- ✓ Test, scan, and monitor.
- ✓ Think production from day one!



GOLDEN RULE

A GREAT CONTAINER IS SMALL, SECURE, RELIABLE, AND EASY TO MAINTAIN.



FINAL DOCKER PROJECT

CONTAINERIZE AN APPLICATION

BUILD YOUR FIRST COMPLETE DOCKER PROJECT FROM START TO FINISH

STEP-BY-STEP PROJECT ROADMAP

- 1 CREATE A SIMPLE WEB APPLICATION**
Build a basic app (e.g., Node.js + Express that shows "Hello Docker!")
- 2 WRITE THE DOCKERFILE**
Define how your application image should be built.
- 3 BUILD THE IMAGE**
docker build -t myapp:1.0 .
- 4 TAG THE IMAGE**
docker tag myapp:1.0 mydockerhub/myapp:1.0
- 5 RUN THE CONTAINER**
docker run -d --name myapp myapp:1.0
- 6 PUBLISH THE APPLICATION PORT**
docker run -d -p 8080:3000 --name myapp myapp:1.0
- 7 ADD ENVIRONMENT CONFIGURATION**
Use environment variables for configuration.
Example: NODE_ENV=production
- 8 ADD PERSISTENT STORAGE**
Mount a volume for data that must persist.
-v mydata:/app/data
- 9 ADD A SECOND SERVICE**
Add a database (PostgreSQL) as a separate container.
- 10 CREATE A COMPOSE FILE**
Define services, volumes, networks, and environment variables in compose.yaml.
- 11 CONNECT SERVICES THROUGH A NETWORK**
Use a custom network so containers can talk to each other by service name.
- 12 INSPECT LOGS**
docker compose logs -f
Check logs to understand app behavior.
- 13 TROUBLESHOOT A DELIBERATELY BROKEN CONTAINER**
Fix port, env, or code issues and restart.
- 14 REBUILD AFTER A CODE CHANGE**
Update code, rebuild the image, and recreate the service.
- 15 PUSH THE FINISHED IMAGE TO A REGISTRY**
docker push mydockerhub/myapp:1.0
- 16 FINAL ARCHITECTURE DIAGRAM**
Your complete multi-container application running in Docker.

EXAMPLE DOCKERFILE (NODE.JS APP)

```
1 FROM node:20-alpine # Small base image
2 WORKDIR /app # Set working directory
3 COPY package*.json ./ # Copy dependency files
4 RUN npm install # Install dependencies
5 COPY . . # Copy application code
6 ENV NODE_ENV=production # Environment variable
7 EXPOSE 3000 # App listens on port 3000
8 USER node # Run as non-root user
9 CMD ["node", "server.js"] # Start the application
```

EXAMPLE COMPOSE.YAML (TWO-SERVICE APP)

```
version: '3.9'
services:
  web:
    build: . # Build from Dockerfile
    ports:
      - "8080:3000" # Host:Container
    environment:
      - NODE_ENV=production
      - DB_HOST=db
    depends_on:
      - db
    networks:
      - app-network # Database service
  db:
    image: postgres:15
    environment:
      - POSTGRES_USER=appuser
      - POSTGRES_PASSWORD=secret
      - POSTGRES_DB=appdb
    volumes:
      - db_data:/var/lib/postgresql/data # Persistent data
    networks:
      - app-network
volumes:
  db_data:
networks:
  app-network:
    driver: bridge
```

USEFUL DOCKER COMMANDS

MANAGE STACK

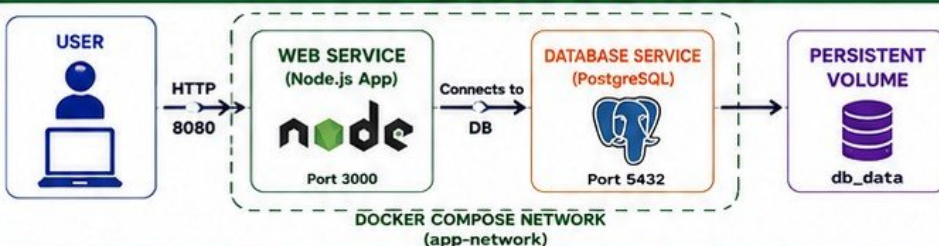
- docker compose up -d
- docker compose down
- docker compose ps
- docker compose logs -f
- docker compose restart



INSPECT & DEBUG

- docker inspect <container>
- docker exec -it <container> sh
- docker network inspect <net>
- docker logs <container>
- docker stats

FINAL ARCHITECTURE DIAGRAM



PROJECT OUTCOME

- ✓ Multi-container application running together
- ✓ Containers communicate using service names
- ✓ Data persists even if containers are removed
- ✓ Easy to start, stop, and recreate the entire stack
- ✓ Production-like workflow with Docker Compose



YOU DID IT!

You have built, run, connected, and managed a complete multi-container application with Docker!



**KEEP BUILDING.
KEEP LEARNING.**